

The Complete Guide to **Caching**



Every layer, pattern, invalidation rule and failure mode you can be asked about in an HLD / system design interview — explained in plain words, with diagrams and model answers.

Where caches live

Browser → CDN → gateway → app → shared cache → DB buffer pool. Which layer solves what.

6 caching patterns

Cache-aside, read-through, write-through, write-behind, write-around, refresh-ahead.

The famous failures

Stampede, penetration, avalanche, hot key / celebrity, big key, cold start — and the fixes.

Sizing & the interview

Working-set maths on a whiteboard, a repeatable framework, 15 model answers and the red flags.

What's inside

Read it front to back once to build the mental model. After that, Part 5 (failure modes) and Part 7 (interview answers) are the pages you revise the night before.

1

Where to put a cache

Seven layers from the browser to the database buffer pool — and how to choose.

2

Caching patterns

Cache-aside, read-through, write-through, write-behind, write-around, refresh-ahead.

3

Invalidation & consistency

TTL vs event-driven, the delete-vs-update race, delayed double delete, CDC invalidation.

4

Eviction policies

LRU, LFU, FIFO, TTL, ARC, W-TinyLFU — and how real caches approximate them cheaply.

5

Cache failure modes

Stampede, penetration, avalanche, hot key / celebrity, big key, pollution, cold start.

6

Capacity planning

Sizing memory, QPS, bandwidth and shard counts on a whiteboard in two minutes.

7

The interview itself

A repeatable framework, 15 asked-in-real-interviews Q&A, and the red flags to avoid.

★

One-page cheat sheet

The night-before revision page: formulas, default decisions, failure modes, caching in eight facts.

This volume is about caching, not about any one cache server. Redis itself — its internals, its data structures, its persistence and failover model and its uses beyond caching — is a separate volume: *The Complete Guide to Redis*.

How to use this guide

Understand

Every section opens with a plain-English explanation before any jargon. If a sentence needs a diagram, there is one.

Say out loud

Grey “interview line” boxes give you the exact sentence that earns the point. Practise saying them.

Defend

Every choice lists its trade-off. Interviewers score you on the trade-off, not the choice.

A NOTE ON CODE

System design interviews test **reasoning, not syntax** — so this volume explains mechanisms in words, steps and diagrams rather than in code. Where a command or a header matters it is named inline. The engine-specific material — Lua scripts, command costs, cluster mechanics — belongs to the companion Redis volume and is deliberately not repeated here.

THE ONE THING TO INTERNALISE

A cache is a **bet**: you trade correctness (data may be stale) and memory (a second copy) for latency and load reduction. Every caching question in an interview is really asking “*do you understand what you gave up?*”

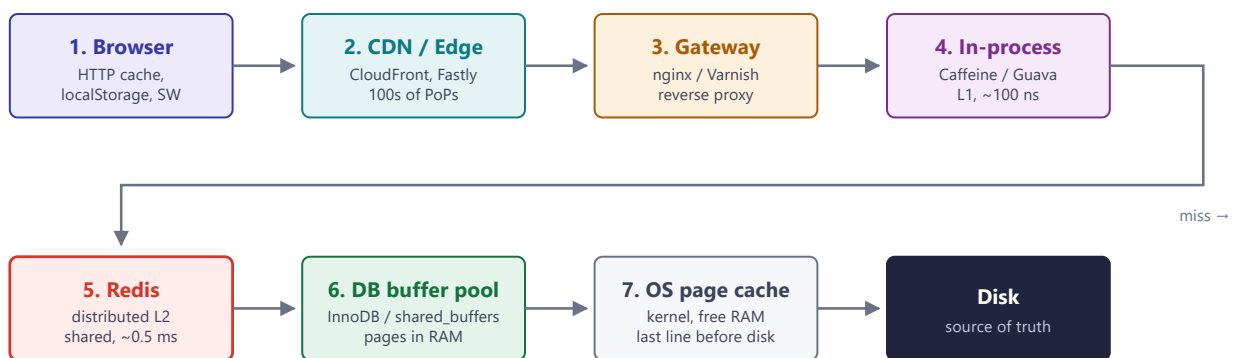
Where to put a cache

PART 1

“Where would you cache?” is the most common opening caching question. The strong answer is not “Redis” — it is “*at several layers, for different reasons,*” followed by naming them.

1.1 The seven layers

Every request travels through a corridor of possible caches. Each one that hits saves everything downstream of it. **The earlier you hit, the more you save** — but the earlier the layer, the less control you have over invalidating it.



What each layer uniquely saves when it hits

- Browser** — the entire network. Zero cost, zero server load. But you cannot invalidate it; only expire it.
- CDN** — the long-haul network hop (up to 150 ms) plus all origin capacity. Invalidate by explicit purge.
- Gateway** — all application compute. 1,000 identical requests become 1 backend call.
- In-process** — even the Redis network hop. Fastest possible, but each server has its own copy (coherence problem).
- Redis** — the database query. Shared by all servers, so one warm-up serves the whole fleet.
- Buffer pool** — disk I/O. Free and automatic; you influence it only by sizing RAM.

The trade-off that runs through the whole diagram:

Left = faster + cheaper, but harder to invalidate and more copies to keep coherent.
Right = slower, but single copy and easy to keep correct. Push data left only as far as your staleness budget allows.

Figure 1.1 — The seven caching layers. Naming these in order is a complete answer to “where would you cache?”

Layer	Latency	Best for	How you invalidate	Main risk
Browser	0 (local)	Static assets, immutable content	Impossible — only expiry or a URL change	Shipping a bug with a long max-age
CDN	10–50 ms	Global static + short-TTL dynamic	Purge API / surrogate keys	Cache-key explosion; leaking private data
Gateway	1–5 ms	Identical anonymous API responses	TTL + purge	Caching personalised responses by mistake
In-process	~100 ns	Hot keys, config, feature flags	Short TTL or Pub/Sub broadcast	Coherence drift; heap pressure & GC
Redis	0.2–1 ms	Shared objects, sessions, computed results	Explicit DEL / TTL — full control	Becomes a critical dependency & SPOF
Buffer pool	~100 ns	Everything, automatically	Automatic, transactional	Under-provisioned RAM

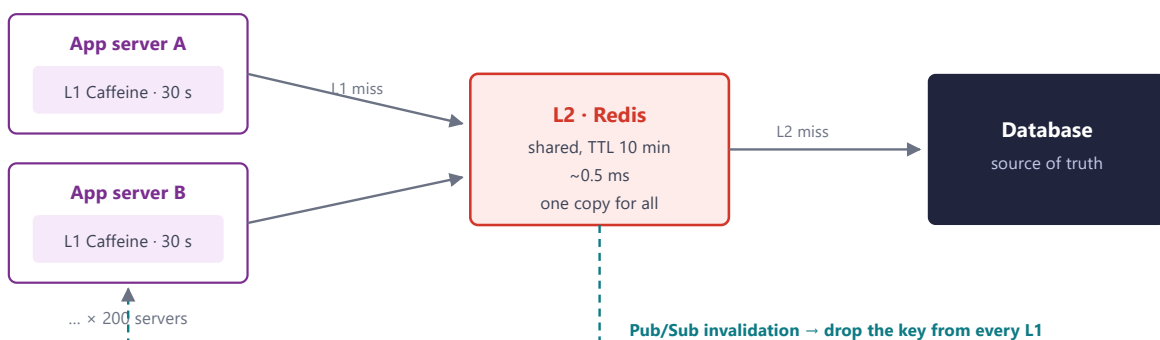
THE BROWSER-CACHE QUESTION INTERVIEWERS ACTUALLY ASK

“How do you invalidate a browser cache?” — **you can't**. If you ship a one-year `max-age` on `/app.js` and then find a bug, those users are stuck for a year. The answer is **content hashing**: name the file `/app.a3f19b.js` and cache it forever, while the HTML referencing it is never cached. Change the content → change the URL → the old entry becomes *irrelevant* rather than *wrong*.

Two related header facts worth knowing: `no-cache` does **not** mean “don't cache” — it means “store it but revalidate every time” (the one that means don't store is `no-store`); and `stale-while-revalidate` serves the stale copy instantly while refreshing in the background, which removes the latency cliff at expiry.

1.2 The combination that matters: L1 + L2

Serious systems run an in-process cache *and* Redis. This is a **multi-level** or *near cache*, and describing it correctly is a senior signal — it is also the standard answer to the hot-key problem in Part 5.



Effective hit ratio: if L1 hits 60% and L2 hits 90% of the remainder, only 4% of requests reach the database — and a viral key is absorbed almost entirely by L1.

Figure 1.2 — Near-cache topology. L1 gives speed and hot-key absorption; L2 gives sharing and a high global hit ratio.

Why you want L1

It absorbs hot keys before they reach Redis. One viral key is served from every app server's own RAM, so Redis sees a few hundred requests per TTL instead of a million per second.

What it costs

Every server has its own copy, so invalidation becomes a broadcast problem. 200 servers means 200 slightly different views of the truth. A short TTL is the pragmatic answer; Redis Pub/Sub is the precise one.

INTERVIEW LINE

"I'd run a two-level cache: a short-TTL in-process L1 to absorb hot keys and save the network hop, backed by Redis as a shared L2 so a cold app server doesn't hammer the database. I'd keep the L1 TTL small — tens of seconds — because it's the layer I can't invalidate precisely, and add Redis Pub/Sub invalidation if the staleness budget is tighter than that."

1.3 Choosing a layer

THE RULE THAT RESOLVES MOST "WHICH LAYER?" DEBATES

Push a piece of data **as far toward the user as its staleness budget allows**. A product image can live in the browser for a year. A price can live in a CDN for 60 seconds. A stock count can live in Redis for 5 seconds. An account balance lives only in the database. **Ask for the staleness budget first, and the layer chooses itself.**

One cautionary tale worth citing here: the **MySQL query cache was removed in 8.0** because invalidating it on any table write made it a global contention point. It is the cleanest proof that *a cache with a bad invalidation model is worse than no cache* — which is the subject of Part 3.

The caching patterns

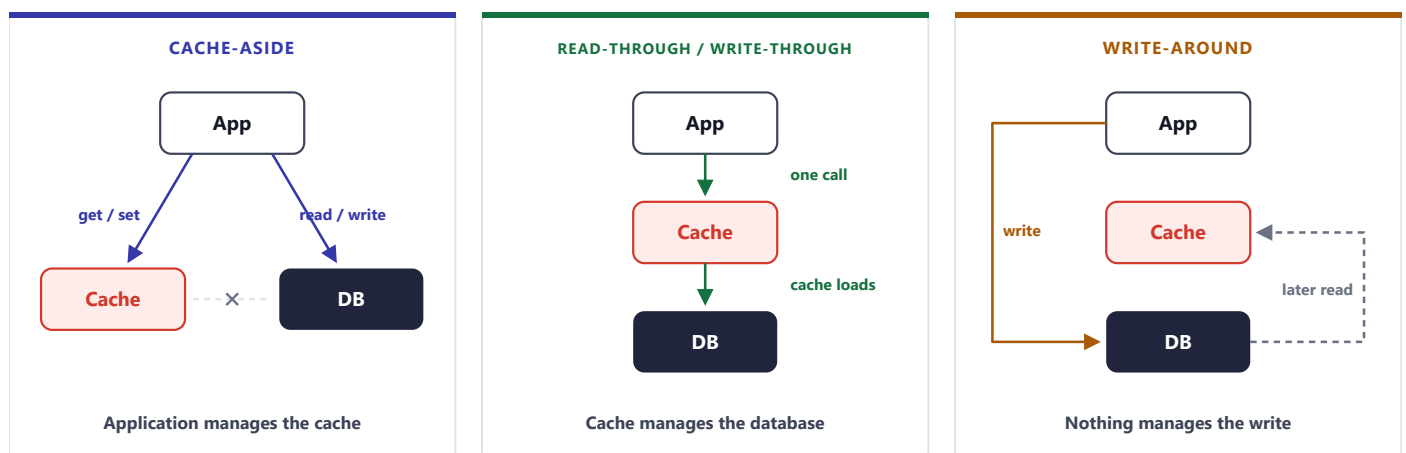
Part 2

Six named patterns. Reads are governed by cache-aside / read-through / refresh-ahead; writes by write-through / write-behind / write-around. Real systems combine one read pattern with one write pattern.

Start here: who talks to whom

Before the names, one question. On a cache miss, who goes to the database — your application, or the cache itself? That question is the whole difference between cache-aside and read-through. Asked about writes instead, the same question separates write-around from write-through and write-behind. **Learn the six patterns as answers to one question and you will never mix them up.**

It matters because it decides where the logic lives, and therefore how much you control. If your code handles the miss, you choose the TTL, you choose whether to cache a not-found, you choose what happens when the database is unavailable. If the cache handles the miss, all of that moves inside a library or a provider: you write less code and you decide less.



The whole taxonomy in one question: on a miss, who goes to the database?

Cache-aside answers "the application". Read-through and write-through answer "the cache". Write-around answers "nobody, until somebody reads it". Everything else in this part is a detail of when, what and for how long.

Figure 2.1 — Three topologies, three owners of the miss. Get the shape right and the names follow.

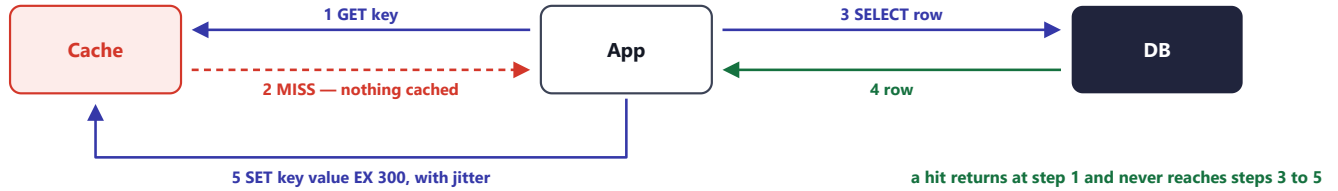
The two sentences to memorise

Cache-aside — the application manages the cache. It talks to the cache, and it talks to the database, and it keeps the two in step itself. **Read-through and write-through — the cache manages the database.** The application makes one call and the cache layer does the rest. Say which side of that line a pattern sits on and its strengths, its weaknesses and its failure modes all follow.

2.1 Cache-aside (lazy loading) — the default

The application talks to *both* the cache and the database. The cache is a dumb key-value store that knows nothing about your database. **This is the pattern to name unless the question pushes you elsewhere — it is what the overwhelming majority of production systems run.**

READ path — a miss, step by step



WRITE path — database first, then delete



Write rule: DELETE the key, do not overwrite it. Then the next read repopulates from the database.

Deleting is idempotent and race-tolerant; writing a value computed from a possibly-stale read is how you poison a cache permanently. Note where the application sits in both rows — in the middle, holding both conversations. That position is the pattern.

Figure 2.2 — *Cache-aside. The application owns all the logic; the cache never talks to the database.*

Read: four steps

1. Ask the cache for the key.
2. If it is there, return it. Done.
3. If not, read from the database.
4. Write it into the cache with a **jittered TTL**, then return it.

Write: two steps

1. Update the database — it is the source of truth, so it goes first.
 2. **Delete** the cache key. Do not write the new value into it (§3.3 explains why).
- The next read will naturally repopulate it.

Two details in step 4 that interviewers listen for: the TTL should carry **random jitter** so keys don't all expire together (§5.3), and you should decide explicitly what to do when the database returns nothing — caching that “not found” result is what protects you from cache penetration (§5.2).

Strengths

Only requested data is cached, so memory tracks the real working set.

Cache failure is survivable — you fall back to the database.

Cache and DB models can differ freely (cache a rendered DTO, not a row).

Dead simple; no special cache infrastructure required.

Every policy decision stays in your hands.

Weaknesses

Every first read is a miss → cold-start penalty after deploys.

Caching logic is duplicated in every call site (fix with a decorator or an AOP layer).

Vulnerable to stampede: N concurrent misses all query the DB.

A read/write interleaving can leave a stale value cached — see §3.3.

Invalidation, and the read that follows

Suppose a price changes. You update the row, then you delete the key. You do not compute the new value and write it in — that is the tempting move and it is the wrong one, because the value you computed may already be stale by the time you store it, and a stale value carrying a fresh TTL is a lie with a guaranteed lifetime. Deleting leaves an **absence**, and an absence can only be filled from the source of truth.

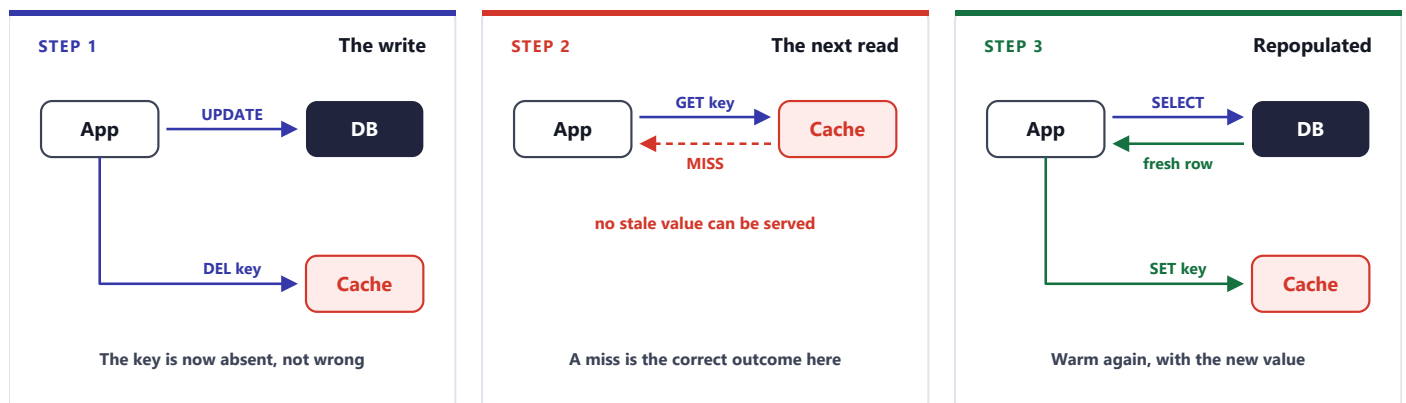


Figure 2.3 — Cache invalidation in cache-aside. The write removes the key; the next reader pays one miss and rebuilds it.

What cache-aside lets you decide

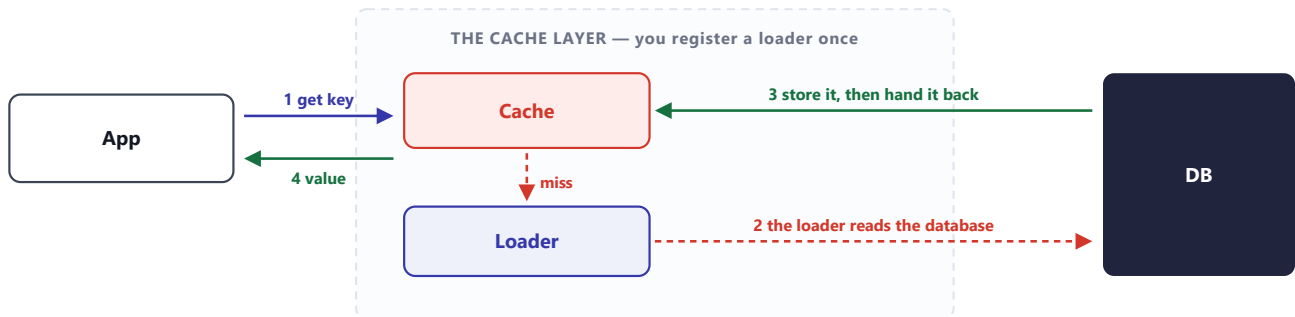
- **What** to cache — a row, a rendered DTO, a whole response.
- **When** to cache — every miss, or only above a read threshold.
- **How long** — a TTL per key class, with jitter.
- **What to invalidate** — one key, a tag, a prefix.
- **Negative caching** — cache a not-found, briefly.
- **Failure policy** — serve stale, or fail the request.
- **Bypass** — route one user past the cache after their own write.
- **Staleness budget** — per field, not per system.

Read-through hands you none of those decisions by default. That is the trade in one line: cache-aside is more code and more control, read-through is less code and less control.

2.2 Read-through

Same read behaviour as cache-aside, but the **cache library** does the loading, not your code. The application only ever calls the cache, and the cache knows how to reach the database.

You register a **loader function** once — “here is how to fetch a product if you do not have it” — and from then on your code only ever calls `cache.get(id)`. The database disappears from the call site entirely. Cache-aside puts the miss-handling in *your* code; read-through puts it in the *cache's* code.



The call site is one line, and it never names the database
`value = cache.get(key)`

Concurrent misses for one key are coalesced into a single load
so read-through is stampede protection as well

Figure 2.4 — Read-through. Steps two and three happen inside the cache layer, where your code cannot see them.

The important practical difference: good read-through implementations **coalesce concurrent loads for the same key** automatically (Caffeine does; so does Guava). That gives you stampede protection for free, which is why naming read-through as a stampede mitigation is legitimate.

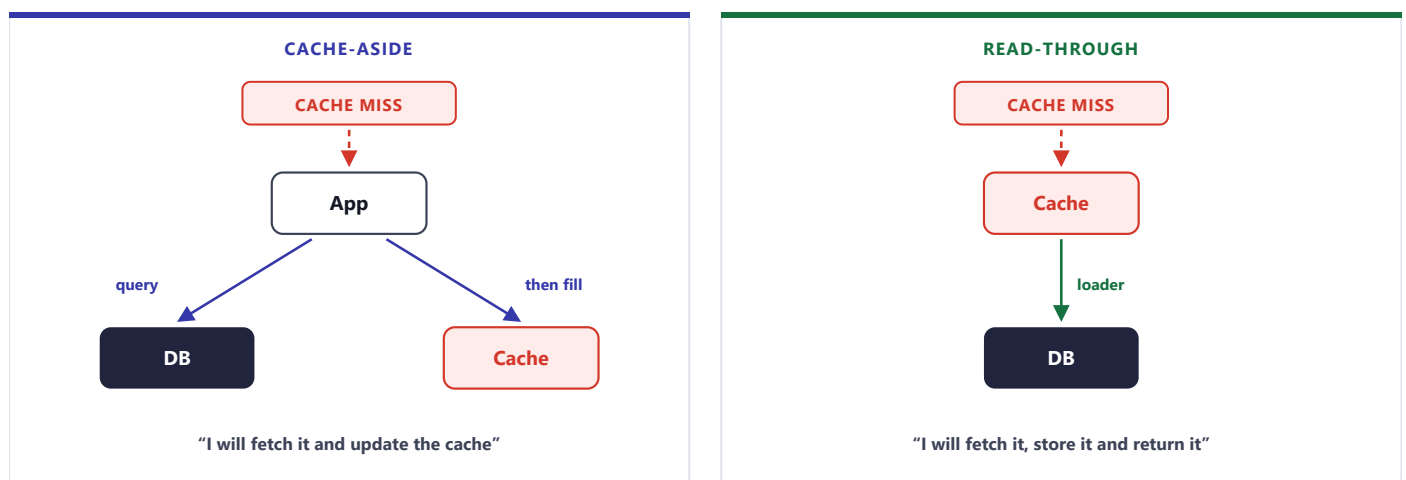


Figure 2.5 — The single difference, drawn. In one the application is on the miss path; in the other it is not.

Cache-aside against read-through, line by line

Both patterns are read patterns, both give the application a cache to call, and both end up with the value in memory. Everything they differ on follows from one row of this table — the fourth.

	Cache-aside	Read-through
Application calls the cache	Yes	Yes
Application queries the DB on a read	Yes	No
Cache knows how to reach the DB	No	Yes
A miss is handled by	The application	The cache layer
Loading logic lives	In every call site, or one shared helper you wrote	Registered once, inside the cache
Concurrent misses on one key	All reach the DB unless you add a lock	Coalesced into a single load
Control of TTL, negative caching, fallback	Complete	Whatever the provider exposes
How often you meet it in production	Very common — the default	Less common — mostly in-process caches

Cache-aside — the miss is yours

```
value = cache.get(key);  
if (value == null) {  
    value = db.load(key);  
    cache.set(key, value, TTL);  
}  
return value;
```

The highlighted block is the pattern. It sits in your service, so **you** own the TTL, the negative caching and the fallback.

Read-through — the miss is the cache

```
// once, at startup  
cache = builder  
    .expireAfterWrite(300s)  
    .build(key -> db.load(key));  
  
// every call site, forever  
return cache.get(key);
```

The same block exists — it moved into the builder. The call site no longer mentions a database.

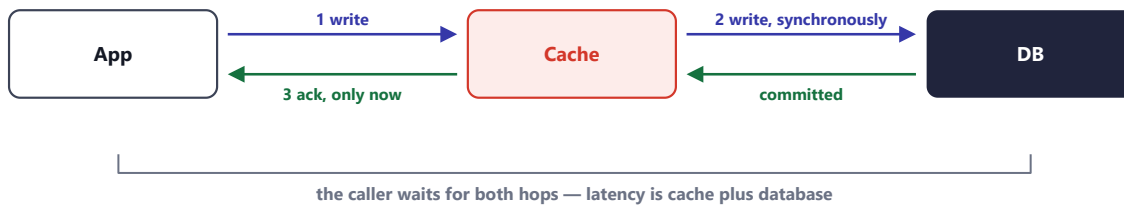
And do not confuse read-through with write-through. Read-through is about the miss path; write-through is about the write path. They are independent choices about opposite directions of travel, and a cache can do both — see §2.8.

2.3 Write-through

Every write goes to the cache, and the cache synchronously writes to the database before acknowledging. Cache and database are updated together, so a reader can never see a cached value the database has not yet accepted.

Notice the shape: application → cache → database. It is the same chain as read-through, walked in the other direction — which is exactly why the two compose so neatly, and also why people confuse them. The boxes are identical; only the direction of travel and the meaning of step two change.

Every write goes through the cache and waits for the database



Cache is never stale

It holds what the database holds

No write is ever lost

A cache crash costs you nothing

Writes pay for it

And cold rows land in memory

Figure 2.6 — *Write-through. Synchronous, safe, slower — the acknowledgement waits for the slowest hop.*

The write-through subtlety worth mentioning

Write-through guarantees the cache is never stale, but it **pollutes the cache with data nobody reads**. If 90% of written records are never read back, you have filled memory with cold data and evicted hot data. The standard fix is to pair write-through with a TTL, or to switch to **write-around** for write-heavy-read-light entities. Saying this shows you understand that write-through is not strictly better than cache-aside.

Reach for it when

A read straight after a write must be correct.
Writes are infrequent relative to reads.
The written row is the row people then read.

Avoid it when

Write latency is on a user-facing path you are defending.
Most written rows are never read again.
The database is the slow dependency you were caching to avoid.

2.4 Write-behind (write-back)

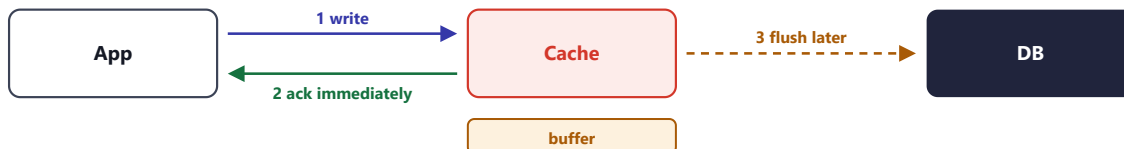
Write to the cache, acknowledge instantly, and flush to the database asynchronously in batches. This is how CPU caches and OS page caches work, and it is the highest-throughput write pattern there is.

Why it wins: it coalesces. A view counter incremented 10,000 times becomes one UPDATE that sets the value to 10,000. That is a genuine ten-thousand-fold reduction in database writes, not a two-fold one.

Why it frightens people: acknowledged data lives only in volatile memory. A crash between the acknowledgement and the flush loses writes you already told the user had succeeded.

How real systems make it safe: let a durable log hold the buffer instead of the cache. Write to Kafka, acknowledge from there, and have consumers batch into the database. You keep the coalescing and lose the data-loss window. Naming this upgrade is a strong senior signal.

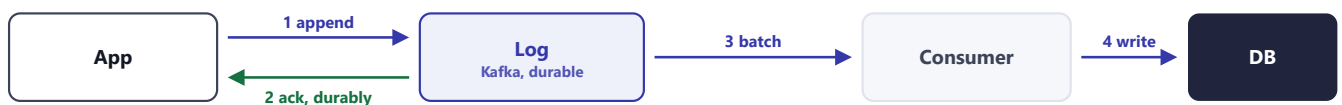
Write-behind — acknowledge first, flush later



Ten thousand increments coalesce into one UPDATE

A crash before the flush loses acknowledged writes

The upgrade an interviewer waits for — put the buffer in a durable log



Same coalescing, no data-loss window. This is what production write-behind actually looks like.

Figure 2.7 — *Write-behind and its safe form. The only real change is where the buffer lives.*

Where plain write-behind is legitimate as-is

- View counts and like counts.
- Metrics and analytics events.
- Last-seen and last-active timestamps.
- Anything where a few lost seconds is a rounding error, not an incident.

2.5 Write-around

Writes go straight to the database and skip the cache entirely; the cache is populated only on read misses. Use it when written data is rarely read soon after — logs, audit records, event ingestion, bulk imports. It avoids cache pollution but guarantees that the first read after any write is a miss.

Writes skip the cache entirely; only reads ever fill it

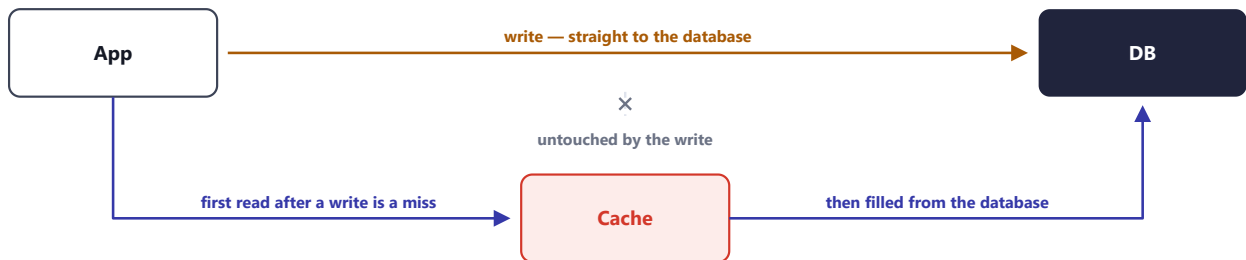


Figure 2.8 — Write-around. Nothing cold is ever written into memory, and you pay exactly one miss per newly written row.

2.6 Refresh-ahead (proactive refresh)

The cache asynchronously refreshes entries before they expire, for keys it predicts will be needed. Caffeine's `refreshAfterWrite` is exactly this: once a value is past the refresh window, the first reader gets the current value immediately while a background thread reloads it. Readers never wait on a miss.

One key on a timeline — the refresh window sits inside the expiry window

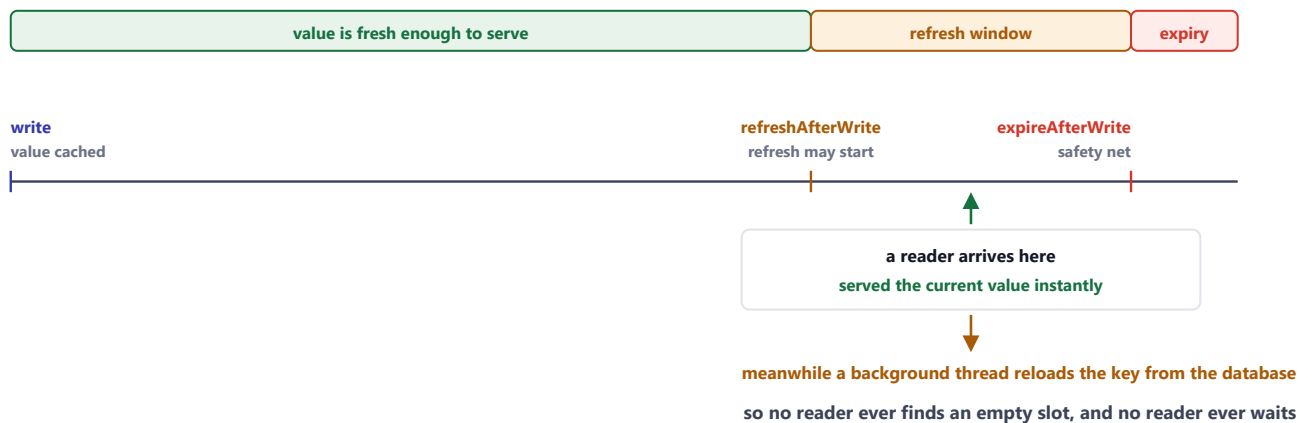


Figure 2.9 — Refresh-ahead. The key never appears missing to a reader, which is why it removes the stampede rather than surviving it.

Why this matters for interviews

Refresh-ahead is the cleanest **structural** answer to cache stampede for a known-hot key set: if the value never actually appears missing from a reader's point of view, there is no moment where 10,000 requests find an empty slot. Combine `refreshAfterWrite(20s)` with `expireAfterWrite(300s)` — refresh keeps it warm, expiry is the safety net if the key goes cold.

2.7 Choosing a pattern

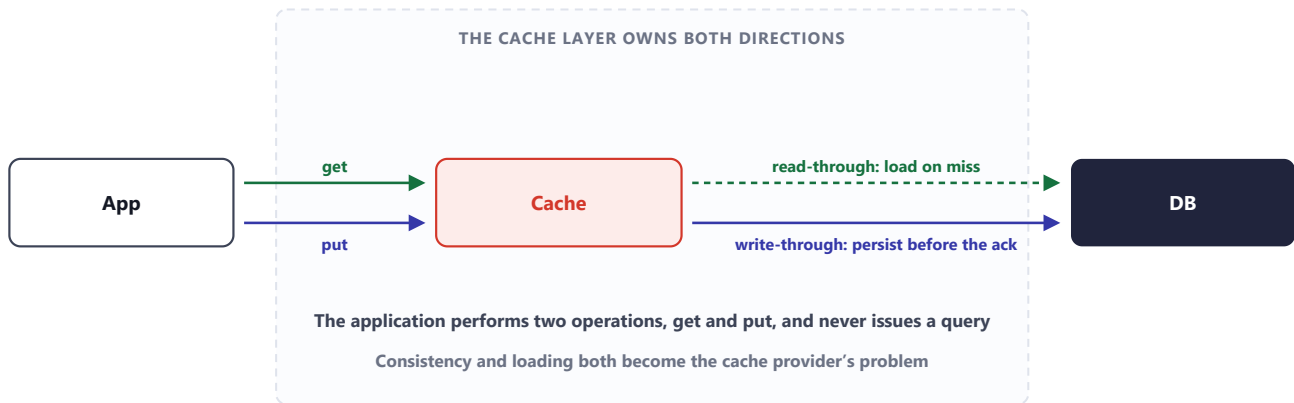
Pattern	Read lat.	Write lat.	Consistency	Data-loss risk	Use when
Cache-aside	Fast	Fast	Eventual	None	Default. Read-heavy, tolerant of brief staleness.
Read-through	Fast	n/a	Eventual	None	You want loading logic centralised and coalescing for free.
Write-through	Fast	Slow	Strong (cache=DB)	None	Read-after-write must be correct; writes are infrequent.
Write-behind	Fast	Fastest	Weak	Real	Counters, metrics, huge write volume, loss tolerable.
Write-around	Miss first	Fast	Good	None	Write-heavy, read-rarely. Avoids pollution.
Refresh-ahead	Fastest	n/a	Eventual	None	Predictably hot keys; stampede-sensitive.

Interview line

"I'd use cache-aside for reads with a TTL, and on writes I'd update the database then delete the key rather than update it — deletion is race-tolerant. For the view-counter path specifically I'd switch to write-behind through Kafka, because there the write volume is the problem and a few seconds of lag is acceptable." — that single sentence covers three patterns and their justifications.

2.8 Combining a read pattern with a write pattern

The six patterns are not six alternatives. Three of them answer “how do I read?” and three answer “how do I write?”, so every real system picks one from each column. The pairing you choose follows from the same ownership question the part opened with.

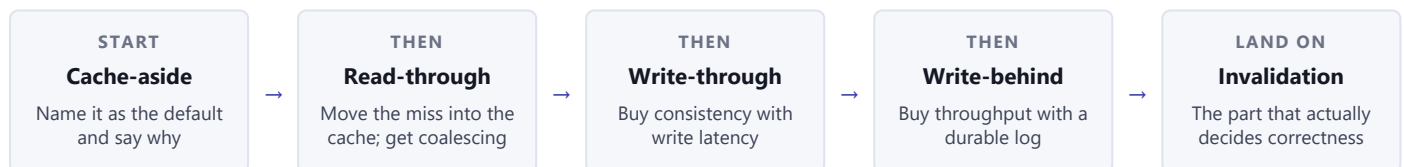


Read-through plus write-through is the classic combination. Cache-aside plus write-around is the other one, and by far the more common.

Figure 2.10 — Combining a read pattern with a write pattern. Read-through and write-through compose into a single cache-managed path.

Read-through with write-through is the classic cache-managed pairing: the application sees a single object with get and put, and the provider keeps it consistent with the database. Cache-aside with write-around, or cache-aside with a delete on write, is the application-managed pairing — and it is the one you should reach for first, because it survives a cache outage and lets you tune every decision per key class.

The order to walk through them in an interview



That progression works because each step is a considered trade rather than a new fact. It also lands you exactly where the interviewer wants to go next, which is Part 3.

Invalidation & consistency

PART 3

"There are only two hard things in computer science: cache invalidation and naming things." — Phil Karlton. This part is why the joke is true, and what to actually do about it.

3.1 Three ways to invalidate

1. TTL (time-based)

Let it expire. **Simple, self-healing, and by far the most common.**

Staleness is bounded by the TTL and nothing can leak forever. Cost: you serve stale data for up to the TTL, and you re-fetch data that never changed.

2. Explicit (write-driven)

On write, delete the key. Precise and immediate. Cost: every write path must know every key that derives from it — and it will eventually forget one. Hard for derived/aggregated data.

3. Event / CDC-driven

Tail the database changelog (binlog / WAL) with Debezium → Kafka, and let a consumer invalidate. **Catches every write, including ones from jobs, migrations and other services.** Highest fidelity, most infrastructure.

WHY CDC-BASED INVALIDATION IS THE MATURE ANSWER

Explicit invalidation fails silently the moment *anything else* writes to the database — a batch job, an admin tool, a data migration, another microservice. CDC hooks the invalidation to the database's own commit log, so it is impossible to write without triggering it. If an interviewer pushes on "what if another service updates the row?", this is the answer they are fishing for.

3.2 TTL: pick it deliberately

The number should come from the business, not from habit:

Data	Typical TTL	Reasoning
Static assets (hashed)	1 year	Content-addressed; the URL changes when the content does.
User profile	5–30 min	Changes rarely; a stale display name is harmless.
Product catalogue	1–10 min	Merchandising changes slowly.
Price	10–60 s	Showing a wrong price has legal/financial consequences.
Inventory count	1–5 s, or none	Oversell risk. Often better to read the DB and reserve atomically.
Session / auth token	= session length	TTL is the feature. Sliding expiry on each request.
Feature flags / config	10–60 s	Must propagate quickly to be useful for incident response.
Search / recommendations	5–60 min	Expensive to compute, no single correct answer anyway.
Account balance	never cache	Correctness is the product.

3.3 The race conditions (this is where interviews get hard)

Two operations, two orderings, and both orderings have a failure mode. You need to know which is *less* bad and why.

Option A — update DB, then delete cache RECOMMENDED

The (rare) failure window for “DB first, then delete”

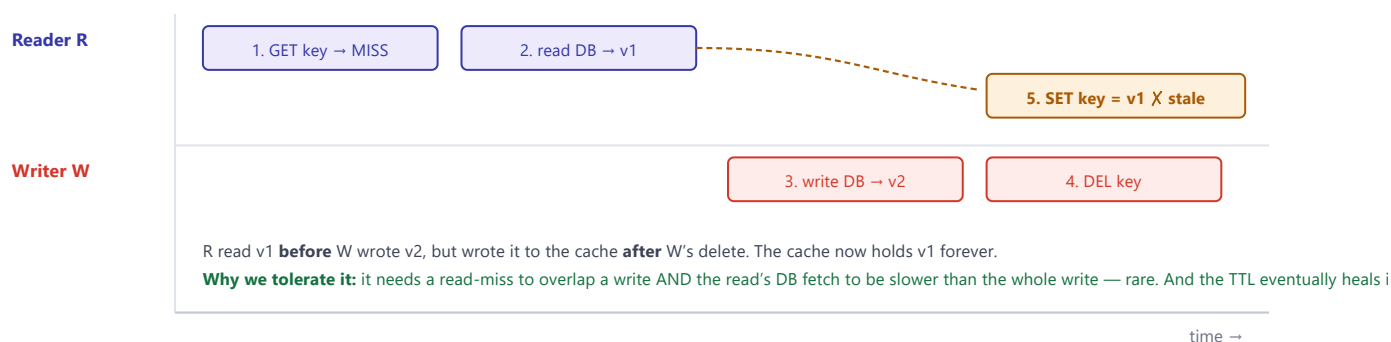


Figure 3.1 — Even the recommended ordering has a race. The defence is a TTL, not cleverness.

Option B — delete cache, then update DB WORSE

Here the bad interleaving is **much more likely**: W deletes the key, then before W's database write commits, R misses, reads the *old* value, and caches it. Now the cache holds stale data and there is no delete coming to fix it. This window is the duration of a database write — milliseconds, hit constantly at scale.

Option C — delayed double delete

The pragmatic patch for Option B, and a term worth knowing because Chinese-authored system design material (and many interviewers) use it:

1. Delete the cache key. 2. Write to the database. 3. Wait a few hundred milliseconds, then **delete the key a second time**.

The second delete removes anything a concurrent reader cached mid-flight. The delay must exceed your slowest read-and-populate. It is a heuristic, not a guarantee — say so when you offer it.

Why “delete” beats “update”

NEVER WRITE THE NEW VALUE INTO THE CACHE FROM THE WRITE PATH

If two writers W1 (value=10) and W2 (value=20) run concurrently, the database order and the cache order are decided independently. The DB can commit W2 last while the cache receives W1 last — leaving **DB=20, cache=10 permanently**. Deleting is idempotent: whoever deletes last, the next read reloads the truth.

“Invalidate, don't update” is one of the highest-value one-liners in a caching interview.

3.4 Making it stronger when you must

Technique	How it works & what it costs
Short TTL	Bounds every failure mode above. Costs hit ratio. Always have one, even with explicit invalidation — it is the backstop for bugs.
Versioned keys	Key includes a version: <code>product:123:v7</code> . A write bumps the version, so old entries become unreachable and expire naturally. No delete race at all — but you need somewhere to keep the version.
Write-through	Cache and DB updated in one path, so they cannot diverge from <i>this</i> service. Slower writes; other writers still bypass it.
CDC invalidation	Binlog/WAL → Kafka → invalidator. Catches every writer. Adds ~100 ms lag and real operational surface.
Read-your-writes	After a user's own write, pin that user's reads to the primary (or write the value to their session) for a few seconds. Fixes the “I edited my profile and it still shows the old name” complaint — which is what stale caches feel like to users.
Leases (Facebook)	On a miss the cache hands out a token; only the token holder may fill the key, and the token is invalidated by any concurrent write. Solves stampede <i>and</i> the stale-set race together. From Facebook's memcache paper — excellent to cite.
Redis client-side caching	RESP3 <code>CLIENT TRACKING</code> : Redis remembers which keys each client cached and pushes an invalidation message when they change. Turns L1 coherence from a guess into a protocol.

INTERVIEW LINE

"I'd treat the cache as eventually consistent and make that explicit: DB first, then delete the key, with a TTL as the backstop so any missed invalidation self-heals within N seconds. If the product needs read-your-writes, I wouldn't try to make the cache strongly consistent — I'd route that user's reads past the cache for a short window instead. And for fields where staleness is genuinely unacceptable, like balances, I wouldn't cache at all."

Eviction policies

PART 4

A cache is finite. When it fills, something must go. The policy that decides *what* goes determines your hit ratio — and this is a favourite whiteboard-coding crossover topic (“implement an LRU cache”).

4.1 The policies

Policy	Evicts	Good at	Fails when
LRU	Least recently used	Temporal locality; the safe default	A full scan (batch job) touches every key once and flushes your hot set
LFU	Least frequently used	Stable popularity; scan-resistant	Popularity shifts — yesterday's viral item has a huge count and won't leave (needs decay)
FIFO	Oldest inserted	Nothing much; simple	Almost always — ignores access entirely
Random	A random key	$O(1)$, no metadata, surprisingly decent	Nothing catastrophic; just mediocre. Redis uses it for <code>allkeys-random</code>
TTL-based	Nearest expiry	When TTL encodes real value	TTL and popularity are unrelated
MRU	Most recently used	Sequential scans over a large file	Rare in practice; know the name
ARC	Adaptive between recency & frequency	Mixed workloads; self-tuning	Patented (historically), more memory per entry
W-TinyLFU	Frequency sketch + small LRU window	Best-in-class hit ratios — what Caffeine uses	More complex; needs a CountMin sketch
Belady / OPT	The item needed furthest in the future	Theoretically optimal	Requires seeing the future. Used only as a benchmark ceiling

LRU vs LFU on the same access stream — capacity 3

Stream: A A A A A B C D (A is accessed 5x, then three one-off keys arrive)

LRU after B, C, D arrive



LFU after B, C, D arrive



But LFU's own trap: if A becomes unpopular tomorrow, its count of 5,000,000 keeps it resident forever.

Real LFU implementations (including Redis's) therefore **decay** counters over time. Frequency without decay is a memory leak of relevance.

Figure 4.1 — The canonical LRU-vs-LFU example. Being able to draw this is usually enough to close the question.

4.2 Implementing LRU (the coding-round crossover)

O(1) get and put via **HashMap + doubly-linked list**. The map gives O(1) lookup; the list gives O(1) reordering. Head = most recent, tail = eviction candidate.

The design, in one breath:

- A **hash map** from key → node gives O(1) lookup.
- A **doubly-linked list** orders nodes by recency. Head = most recently used, tail = the eviction victim.
- On a **get**: look up the node, unlink it, re-insert at the head.
- On a **put** when full: remove the tail node and delete its key from the map, then insert at the head.
- The list must be **doubly** linked so that unlinking a node found via the map is O(1) — with a singly-linked list you would have to walk it to find the predecessor.

In Java you rarely write this: `LinkedHashMap` with access-order enabled and an overridden `removeEldestEntry` is LRU in six lines. The point worth making in an interview is the **concurrency cost**: every read mutates the shared list, so a naive implementation needs a global lock and becomes a bottleneck. That is exactly why real caches (Caffeine, and Redis itself) avoid maintaining exact ordering — which leads directly to the next section.

4.3 How Redis actually does it — approximation

True LRU requires a global linked list updated on *every single access*. At a million ops/second that linked-list maintenance and its 16 extra bytes per key are unaffordable. So **Redis approximates**, and knowing this is a genuine differentiator.

Redis approximated LRU: sample, don't sort

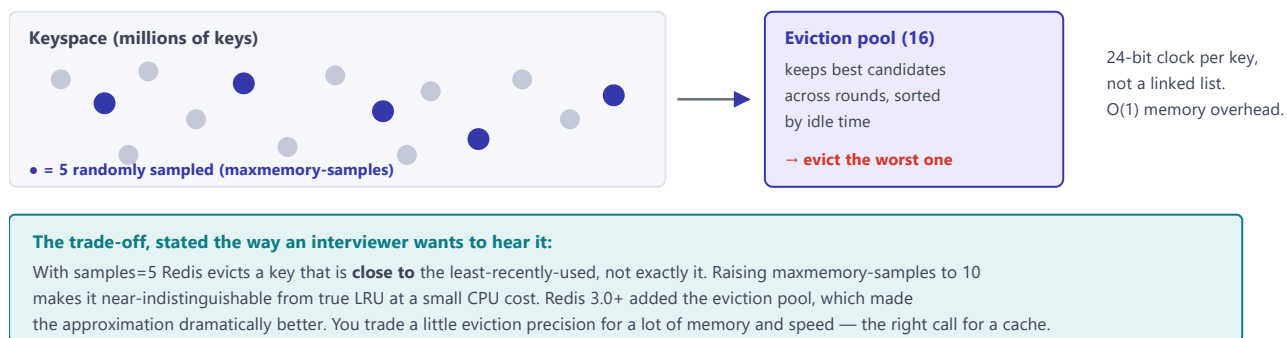


Figure 4.2 — Sampling instead of sorting. The same idea powers Redis's LFU.

Redis LFU is approximated too, and more cleverly. Each key gets 8 bits: a **Morris counter** (a probabilistic counter that increments with decreasing probability as it grows, so 8 bits can represent ~1 million accesses) plus a **decay** driven by `lfu-decay-time`. Two knobs:

lfu-log-factor How fast the counter saturates. Higher = better discrimination among very hot keys.

lfu-decay-time Minutes before a counter halves. This is what stops yesterday's viral item from squatting in memory forever.

4.4 Redis `maxmemory-policy` — all eight values

Policy	Behaviour	When to use
noeviction	Writes fail with OOM error; reads keep working	Default. Correct when Redis is a database/queue, catastrophic when it's a cache
allkeys-lru	Evict least-recently-used from all keys	The standard cache setting. Pick this unless you have a reason not to
allkeys-lfu	Evict least-frequently-used from all keys	Traffic has viral spikes you don't want polluting the hot set
allkeys-random	Evict a random key	Uniform access with no discernible pattern; cheapest
volatile-lru	LRU, but only among keys that have a TTL	Mixed workload: persistent data (no TTL) + cache data (TTL)
volatile-lfu	LFU among keys with a TTL	Same mixed workload, frequency-skewed
volatile-random	Random among keys with a TTL	Rare
volatile-ttl	Evict the key with the nearest expiry	When short TTL genuinely means low value

THE PRODUCTION INCIDENT THIS TABLE PREVENTS

Two traps, both common in real outages:

1. Leaving the default `noeviction` on a cache. Memory fills, and every `SET` starts returning `OOM command not allowed when used memory > 'maxmemory'`. Your cache stops accepting new entries — the hit ratio decays to whatever is already there and the database takes the rest.
2. Choosing a `volatile-*` policy when **most keys have no TTL**. Redis has nothing eligible to evict, so it behaves exactly like `noeviction` and fails writes while sitting on gigabytes of evictable-looking data. Mentioning this specific failure marks you as someone who has actually operated Redis.

The three settings that decide eviction behaviour, and the numbers you watch afterwards:

Setting / metric	What it does or tells you
<code>maxmemory</code>	The ceiling. Eviction does not happen at all until this is set — an unset limit means Redis grows until the OS kills it.
<code>maxmemory-policy</code>	Which of the eight policies above to apply. Set it to <code>allkeys-lru</code> for a cache.
<code>maxmemory-samples</code>	How many random keys to sample per eviction round. Default 5; 10 is close to true LRU at a small CPU cost.
<code>keyspace_hits</code> / <code>_misses</code>	Your hit ratio = hits ÷ (hits + misses).
<code>evicted_keys</code>	Keys thrown out for space. Should be near zero on a correctly sized cache.
<code>expired_keys</code>	Keys removed because their TTL elapsed. Healthy — this is the cache working as designed.

THE METRIC THAT MATTERS MOST

`evicted_keys` climbing steadily means your cache is too small for its working set — you are thrashing, and your hit ratio is being decided by capacity rather than by policy. That is a sizing problem, and no eviction policy will fix it. Watch `evicted_keys` alongside hit ratio, not instead of it.

Cache failure modes

PART 5 · THE INTERVIEW CORE

This is the part interviewers actually probe. Each failure mode has a name, a mechanism, and a small set of accepted fixes. Know all three for each — naming the problem without the fix scores nothing.

The three that get confused with each other

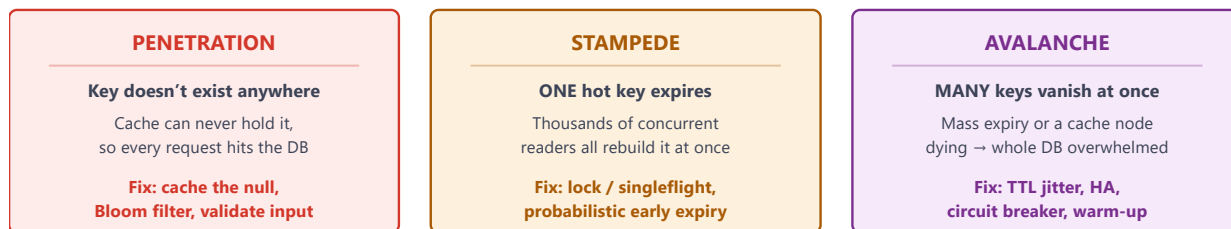
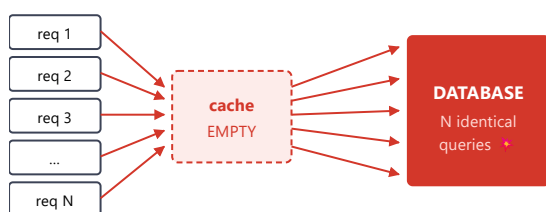


Figure 5.1 — One key that can't exist / one key that expired / many keys gone. Getting these three names right is table stakes.

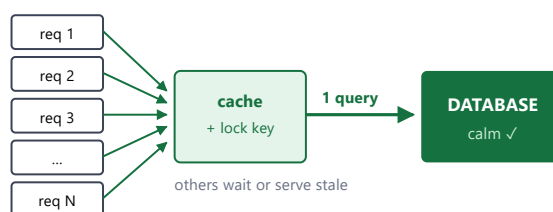
5.1 Cache stampede (a.k.a. thundering herd, dogpile)

A single very popular key expires. In the microseconds before anyone can repopulate it, every concurrent request for that key misses, and all of them run the expensive query simultaneously. A key serving 50,000 req/s that takes 500 ms to rebuild produces **25,000 identical concurrent database queries**. The database saturates, queries slow down, the rebuild takes even longer, more requests pile in — and the system does not recover on its own.

Without protection



With a mutex / singleflight



Note the shape: the fix does not make the rebuild faster. It makes the number of rebuilds equal to one.
Every stampede solution is a variation on “elect a single rebuild, and give everyone else something to do while they wait.”

Figure 5.2 — Cache stampede and the mutex fix.

Fix 1 — Distributed lock (mutex / singleflight)

On a miss, everyone races to acquire a short-lived lock on a key like `lock:product:42`:

- The **one winner** re-checks the cache (someone may have filled it in the meantime), rebuilds from the database, writes the value, and releases the lock.
- Every **loser** either waits briefly and retries the cache, or immediately returns a stale/default value.

Two details that matter: the lock **must have an expiry**, or a crashed rebuild blocks the key forever; and the winner must **re-check the cache after acquiring**, or the second wave of requests rebuilds again needlessly. Note also that within a single

server your cache library already does this — Caffeine coalesces concurrent loads per key, and Go calls the same idea `singleflight`. The distributed lock is only needed to coalesce *across* servers.

Fix 2 — Probabilistic early expiration (XFetch)

The most elegant fix, and naming it well is a strong signal. Instead of everyone treating the key as valid until the exact expiry instant, each reader independently rolls a die that gets progressively more likely to trigger a refresh as expiry approaches. One unlucky reader refreshes *early*, while everyone else still gets a hit — **so the key is never actually empty**.

The mechanism in words:

- Alongside the value, store **how long the rebuild took** and the **absolute expiry time**.
- On every read, the client computes a small random “early window” scaled by that rebuild cost, and asks: *would I have expired if my clock ran that much ahead?*
- Far from expiry the answer is almost always no. As expiry approaches, the probability rises smoothly toward certainty.
- So exactly one unlucky reader tends to refresh early — and **everyone else still gets a cache hit**, because the key never actually became empty.

The elegant part: the window scales with **how expensive the value was to build**. Keys that take 2 seconds to rebuild start protecting themselves much earlier than keys that take 20 ms — the protection is automatically proportional to the risk. This is the XFetch algorithm from the 2015 VLDB paper *Optimal Probabilistic Cache Stampede Prevention*; naming it is a strong signal.

Fix 3 — Logical (soft) expiry

Never let the key expire in Redis at all. Store `{value, logicalExpiryTimestamp}` with no TTL. Readers check the timestamp themselves: if it has passed, one reader takes a lock and refreshes **asynchronously** while *everyone, including that reader, is served the stale value immediately*. No user ever waits and there is never an empty slot. The cost is unbounded staleness if the refresher keeps failing.

Fix 4 — Others

- **Refresh-ahead** (§2.6) — proactively reload before expiry. Structural, not reactive.
- **Request coalescing at the proxy** — nginx `proxy_cache_lock`, Varnish request collapsing. Free if you already run one.
- **Leases** — Facebook's memcache approach: the cache issues a fill token to exactly one client on miss.
- **Never expire the top-N keys** — keep a curated hot set refreshed by a background job.

5.2 Cache penetration

Requests for keys that **do not exist in the database at all**. The cache can never satisfy them — there is nothing to cache — so 100% of them reach the database, every time. Sometimes accidental (a deleted product still linked from Google), often **malicious**: an attacker loops over `/api/user/-1`, `-2`, `-3` ... and walks straight through your cache into your database.

Fix 1 — Cache the null

On a DB miss, store a tombstone with a **short** TTL (30–60 s): `SETEX user:99999 60 "__NULL__"`.

Simple and effective. Two caveats: keep the TTL short so a later real insert becomes visible quickly, and remember an attacker with unbounded random keys can still fill your memory with tombstones.

Fix 2 — Bloom filter

A probabilistic set of *all IDs that exist*, sitting in front of the cache. “Definitely not present” → reject instantly without touching cache or DB. “Possibly present” → proceed normally.

No false negatives, so it is safe to reject on. ~1% false positives at ~10 bits/element: 1 billion IDs in about 1.2 GB.

Bloom filter as a gate in front of the cache



The property that makes this work — and the one that limits it:

A Bloom filter can say “definitely not in the set” with certainty, but “probably in the set” only probabilistically. Since we only act on the definite answer, false positives cost one wasted lookup — harmless. False negatives would be fatal, and cannot occur.

Limitation: a standard Bloom filter cannot delete. Use a Cuckoo filter (RedisBloom: CF.*) if IDs are removed, or rebuild periodically.

Figure 5.3 — Bloom filter gating. `BF.ADD` / `BF.EXISTS` via the RedisBloom module.

Also mention: input validation (reject negative or malformed IDs before any lookup) and per-IP rate limiting. The cheapest defence against a scripted penetration attack is often not a caching technique at all.

5.3 Cache avalanche

A **large fraction of the cache disappears at once**, and the full unprotected load lands on a database sized for 1–10% of it. Two distinct causes, and you should name both:

- **Mass simultaneous expiry** — a warm-up job loaded 500,000 keys at 03:00 with an identical `TTL=3600`, so all 500,000 expire in the same second at 04:00.
- **Cache node / cluster loss** — a Redis node dies, or the whole cluster restarts. Hit ratio goes from 99% to 0% instantly. Since the database only ever saw 1% of traffic, that is a **100× load spike** arriving at a database provisioned for none of it.

TTL jitter

Never use a constant TTL. `ttl = base + rand(0, base×0.2)` spreads expiry over a window instead of a point. One line of code; prevents the entire first cause.

HA + replication

Redis Sentinel or Cluster with replicas so a single node's death is a failover, not an outage. Multi-AZ. This addresses the second cause.

Circuit breaker

When DB latency or error rate spikes, stop sending it traffic. Serve stale, degraded, or an error — a fast 503 beats a total collapse and lets the DB recover.

Request coalescing

The stampede fixes from §5.1 apply per-key during an avalanche too, collapsing millions of misses into thousands.

Warm-up / prewarming

Never route production traffic to a cold cache. Replay recent keys or copy from a peer before adding the node to the pool.

Bulkheads + load shedding

Cap the connection pool to the DB so excess requests fail fast instead of queueing. Shed low-priority traffic first to protect checkout over browsing.

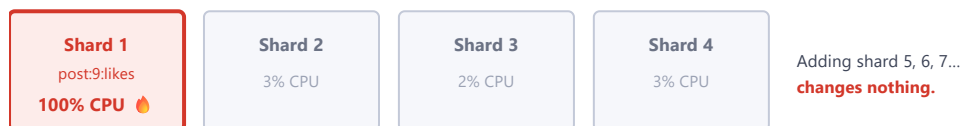
INTERVIEW LINE

"Because my database is provisioned assuming a 99% hit ratio, the cache is no longer just an optimisation — it's a hard dependency. So I'd add jittered TTLs to prevent synchronised expiry, run Redis with replicas and automatic failover, and put a circuit breaker plus a bounded DB connection pool in front of the database so that if the cache *does* vanish, we shed load and degrade instead of taking the database down with it."

5.4 The hot key / celebrity problem

Sharding distributes keys, not load. A key is hashed to exactly one shard, so no matter how many nodes you add, `post:celebrity:likes` lives on *one* of them. When a celebrity with 200 million followers posts, that single node receives the entire spike while the other 99 idle. You cannot scale out of this by adding nodes — which is precisely why it makes such a good interview question.

Problem — one key, one shard, all the traffic



Fix — key splitting (salting): turn 1 key into N sub-keys



Figure 5.4 — Key splitting. Writes fan out across shards; reads pay an N-key gather, or read a periodically-materialised total.

The full toolkit — name several, not one

Technique	How	Trade-off
Local L1 cache	Each app server caches the hot key in-process for a few seconds	The single best answer for hot reads. 1M req/s becomes ~200 Redis reads per TTL. Cost: brief staleness
Key splitting / salting	<code>key:{0..N}</code> , write to a random one, read and merge all N	The answer for hot writes. Reads become N× more expensive
Read replicas	Replicate the hot shard; fan reads across replicas	Helps reads only, not writes. Adds replication lag
Key replication	Store copies as <code>key:copy1..copyN</code> on different shards; readers pick randomly	Spreads read load; N× the invalidation work
Write batching	Buffer increments in the app / Kafka, flush aggregates every second	Turns 1M INCRs into 1 INCRBY. Counter lags by the flush interval
Hot-key detection	<code>redis-cli --hotkeys</code> , <code>MONITOR</code> sampling, client-side counters	Lets you apply a fix <i>before</i> saturation. Operational, not a fix by itself

The sharded counter, in three rules:

- **Write:** pick one of 16 sub-keys at random and increment it. Each write touches a different shard, so the load spreads evenly.
- **Read:** fetch all 16 in a *single* multi-key request and sum them. One round trip, not sixteen.
- **At extreme scale:** stop summing on read. Have a background worker sum the shards once a second into a single total key, and let readers hit that. Reads go back to being O(1) at the cost of one second of lag — which nobody notices on a like count.

THE REDIS CLUSTER DETAIL THAT MAKES THIS WORK

Key splitting only helps if the sub-keys land on **different shards**. Redis Cluster hashes the substring inside `{}` if present — so `post:{9}:likes:0` and `post:{9}:likes:1` hash *identically* and land on the same node, defeating the whole exercise. Use hash tags to *co-locate* keys deliberately (for multi-key operations), and **omit them** when your goal is to spread load. Mentioning this unprompted is a strong Redis-Cluster signal.

5.5 Big key (and hot big key)

A single key holding megabytes — a 5 MB serialised list, a Hash with 10 million fields. Because **Redis is single-threaded**, one big operation blocks *every other client*.

- **Symptoms:** unexplained p99 latency spikes, network card saturation, cluster resharding stalls, `DEL` on a huge key freezing the server for seconds.
- **Find them:** `redis-cli --bigkeys`, `MEMORY USAGE key`, `redis-cli --memkeys`.
- **Fix:** split into sub-keys; use Hash fields instead of one blob so you can `HGET` one field; paginate with `HSCAN` / `ZRANGE` instead of `HGETALL` / `LRANGE 0 -1`; compress values; use `UNLINK` (non-blocking free on a

background thread) instead of `DEL`.

- **Never** run `KEYS *` in production — it is O(N) and blocks the whole server. Use `SCAN` with a cursor.

5.6 Cache pollution and cold start

Pollution / churn

A nightly analytics scan reads every row once, evicting the hot working set with data nobody will read again. Hit ratio collapses at 02:00 and recovers slowly.

Fixes: LFU or W-TinyLFU instead of LRU (scan-resistant by design); route batch jobs to a replica or straight to the DB, bypassing the cache; use a separate cache namespace/instance for analytics.

Cold start

After a deploy, restart or failover the cache is empty — the avalanche scenario, but self-inflicted and scheduled.

Fixes: prewarm from a key log or by replaying recent traffic; bring nodes into rotation gradually so load ramps; use Redis persistence (RDB/AOF) so a restarted node reloads its dataset instead of starting empty — a genuinely underrated argument for enabling persistence on a cache.

5.7 Quick-reference: symptom → diagnosis → fix

Symptom	Likely cause	First moves
Periodic DB spikes at fixed intervals	Synchronised TTL expiry	Add TTL jitter; stagger warm-up jobs
One Redis node at 100% CPU, others idle	Hot key	<code>--hotkeys</code> ; add L1 cache; split the key
DB spike immediately after a deploy	Cold cache	Prewarm; enable persistence; ramp traffic
High DB load but Redis looks healthy	Penetration (missing keys)	Log the missing key pattern; cache nulls; Bloom filter
Latency spikes on one popular endpoint	Stampede on one key	Lock / singleflight; probabilistic early expiry
Hit ratio falling steadily over weeks	Working set outgrew memory	Check <code>evicted_keys</code> ; scale up or shard
Hit ratio collapses nightly	Batch-job pollution	Switch to LFU; isolate the batch workload
Random p99 spikes, no pattern	Big key blocking the event loop	<code>--bigkeys</code> ; split; <code>UNLINK</code> ; audit for <code>KEYS</code>
Users see stale data after editing	Missed / raced invalidation	DB-then-delete ordering; shorter TTL; read-your-writes routing
Writes rejected: OOM error	<code>noeviction</code> , or <code>volatile-*</code> with no TTLs	Set <code>allkeys-lru</code> ; verify keys actually carry TTLs

Capacity planning

PART 6

Interviewers ask “how big does the cache need to be?” to see whether you can reason numerically under pressure. You do not need precision — you need a defensible method and round numbers.

6.1 The four questions to answer

1. How much memory?

working-set entries $\times$ (key + value + overhead) $\times$ replication factor, then add headroom.

2. How much throughput?

Peak QPS $\times$ expected hit ratio = ops the cache must serve.
Compare with $\sim 100\text{k}$ ops/s per Redis node.

3. How much bandwidth?

ops/s $\times$ average value size. This is the constraint people forget, and it bites first on hot keys.

4. How many nodes?

The max of the memory answer and the throughput answer — then add replicas for HA, not for capacity.

6.2 A worked example — product catalogue for an e-commerce site

GIVEN

50 million products $\cdot$ 200,000 read QPS at peak $\cdot$ average serialised product = 2 KB $\cdot$ traffic is heavily skewed: the top 5% of products serve $\sim 90\%$ of reads.

Step	Calculation	Result & reasoning
Working set	$50\text{ M} \times 5\%$	2.5 M products. Cache the working set, not the catalogue — the other 47.5 M would cost 95 GB to serve 10% of traffic.
Raw data	$2.5\text{ M} \times 2\text{ KB}$	5 GB
+ Redis overhead	$\times \sim 1.25$	6.25 GB — key strings, dict entries, expire table, allocator rounding.
+ Headroom	$\times \sim 1.5$	~9.4 GB — fragmentation, growth, and room for <code>fork()</code> copy-on-write if snapshotting.
Node count (memory)	9.4 GB on a 16 GB node	1 primary is enough on memory alone.
Throughput check	$200\text{k QPS} \times 0.9\text{ hit ratio}$	180k ops/s. A single Redis node does roughly 80–120k ops/s comfortably → throughput, not memory, is the binding constraint.
Node count (throughput)	$180\text{k} \div \sim 100\text{k}$	2–3 primaries , or 1 primary + 2 read replicas since this workload is read-only.
Bandwidth	$180\text{k} \times 2\text{ KB}$	~360 MB/s = ~2.9 Gbps. That saturates a 1 Gbps NIC — needs 10 Gbps, or split across nodes. This is usually the real limit.
DB relief	$200\text{k} \times (1 - 0.9)$	20k QPS reaches the database instead of 200k.
Final	—	3 primaries × 16 GB with 1 replica each , multi-AZ. Sized by throughput and bandwidth; memory has plenty of slack.

THE MOMENT THAT EARNS THE POINT

Noticing that **bandwidth and throughput bind before memory does**. Most candidates compute the memory number, say “10 GB, one node,” and stop. Saying “memory says one node, but 180k ops/s and ~3 Gbps say three” demonstrates that you have actually operated a cache rather than only read about one.

6.3 Numbers worth memorising

Redis throughput

~100k ops/s per node. Quote this and note it is conservative — tuned setups approach 1M.

Redis latency

p50 under 1 ms in-datacenter. A much worse p99 means a big key or a slow script.

Per-key overhead

~50–100 bytes beyond your data. Huge for tiny values — bucket them into hashes.

Practical node size

Under ~25 GB. Larger means slow failover, long syncs and dangerous fork pauses.

Memory headroom

Run at ≤50–60% of box RAM if you snapshot — copy-on-write can double usage.

Hit ratio to assume

90% is defensible for a read cache with real popularity skew. State it as an assumption.

IF ASKED HOW YOU'D VERIFY IT IN PRODUCTION

Three metrics: **hit ratio** (is the cache earning its place), **evicted_keys** (is it too small — should be near zero), and **p99 latency** (is something blocking the single thread). A high hit ratio *alongside* heavy eviction means you are thrashing and one traffic shift away from a cliff.

The interview itself

PART 7

Knowing the material is necessary but not sufficient. This part is about *sequencing* — the order in which you say things, so the interviewer hears a designer rather than a list of memorised terms.

7.1 A framework that works every time

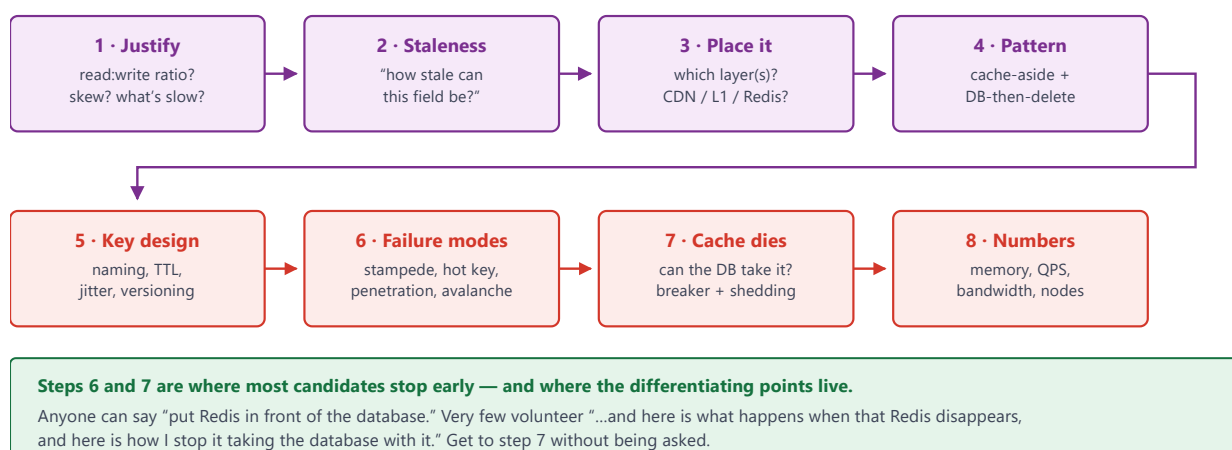


Figure 7.1 — Eight steps. Steps 1–2 take twenty seconds and prevent most wrong turns.

7.2 Key naming — a small thing that signals experience

The convention is a colon-delimited hierarchy: `app : entity : id : field : version`

Key	Verdict	Why
<code>user:1042:profile</code>	Good	Scannable, obvious owner, groups cleanly
<code>product:88:price:v3</code>	Good	The version lets you invalidate an entire generation at once
<code>lb:global:2026-08</code>	Good	Time-bucketed — the whole month expires by itself
<code>u1042</code>	Bad	No namespace — unsearchable, collides, meaningless in a dump
<code>"User Profile 1042"</code>	Bad	Spaces and unstable formatting; easy to generate two variants of the same key
<code>user:1042</code> holding 4 MB	Bad	A big key — blocks the single-threaded event loop for every other client

Mention: a consistent prefix lets you `SCAN MATCH product:*` for debugging, keeps metrics per-namespace, and lets you migrate one entity type at a time. Include a schema version in the prefix (`v3:product:88`) so a serialisation change is a cache miss rather than a deserialisation crash — **this is a genuinely under-mentioned production detail.**

7.3 Fifteen questions with model answers

Q1 What is caching and why do we use it?

Storing a copy of data closer to the consumer so we can avoid recomputing or re-fetching it. Two distinct benefits worth separating: **lower latency** for the user, and **lower load** on the backend. Those are different goals with different targets — latency wants ~90% hit ratio, load protection wants 99%+.

Follow-up trap: “so cache everything?” — No. Name the four cases where caching is wrong: low reuse, correctness-critical reads, write-heavy workloads, and a backend that is already fast (see Q13).

Q2 Where would you place caches in this architecture?

Walk the request path: browser → CDN → API gateway → in-process L1 → shared cache (L2) → DB buffer pool. Then say what each *uniquely* saves, and pick two or three for the actual design with reasons.

Signal: naming the layers *and* declining to use some of them. “I wouldn’t CDN-cache this because it’s per-user.”

Q3 Cache-aside vs read-through vs write-through — which and why?

Cache-aside by default: it is resilient (cache down ≠ site down), memory tracks the real working set, and the cache model can differ from the DB model. Write-through when read-after-write correctness matters and writes are infrequent. Write-behind only for high-volume, loss-tolerant data like counters — and I’d back it with a durable log rather than trusting cache memory.

Q4 On a write, do you update the cache or delete the key?

Delete. Updating from the write path races: two concurrent writers can commit to the DB in one order and to the cache in the other, leaving them permanently divergent. Deletion is idempotent — whoever deletes last, the next read reloads the truth. Order: **database first, then delete.**

Bonus: explain why delete-then-write is worse (§3.3), and offer delayed double delete as the patch if forced into that order.

Q5 What is a cache stampede and how do you prevent it?

One hot key expires and thousands of concurrent readers all rebuild it simultaneously, saturating the database. Fixes, best first: **probabilistic early expiration** (readers randomly refresh before expiry, so the key is never empty), **logical expiry** (serve stale, refresh async), **distributed lock / singleflight** (elect one rebuilder), and **refresh-ahead** for a known hot set.

Signal: noting that every solution reduces the *number* of rebuilds to one rather than making the rebuild faster.

Q6 Difference between cache penetration, stampede and avalanche?

Penetration — key doesn't exist anywhere, so the cache can never help; often malicious. Fix: cache nulls with short TTL, Bloom filter, input validation.

Stampede — one hot key expired; many concurrent rebuilds. Fix: lock, early expiry.

Avalanche — many keys vanish at once (synchronised TTLs or a node dying). Fix: TTL jitter, HA, circuit breaker.

Q7 A celebrity posts and one key gets a million requests per second. What happens?

That key hashes to one shard, so one node absorbs everything while the rest idle — adding nodes does not help. For hot *reads*: an in-process L1 cache with a few seconds' TTL, so 200 servers make ~200 cache calls per TTL instead of a million per second. For hot *writes* (the like counter): **key splitting** — write to `post:9:likes:{0..15}` at random, and have a worker sum them into a total that reads consume.

Killer detail: the split sub-keys must land on *different* shards — that is the entire point — so never co-locate them with a hash tag or a shared prefix that your router hashes on.

Q8 Your cache tier dies completely. What happens to the system?

If the DB was provisioned assuming a 99% hit ratio, it now receives 100× its expected load and will fall over — so the cache is a hard dependency, not an optimisation. Mitigations: replicas with automatic failover; a circuit breaker so we shed load instead of queueing; a bounded DB connection pool so excess requests fail fast; serving stale from L1; and degrading gracefully (approximate data, disable non-essential features).

This is the question that separates senior from mid. Volunteer it before being asked.

Q9 How do you choose a TTL?

From the business, not from habit: ask how stale *this specific field* may be. Then add jitter so keys don't expire in lockstep. Always set some TTL even with explicit invalidation — it is the backstop that makes a missed invalidation self-healing rather than permanent.

Q10 How do you keep in-process (L1) caches coherent across 200 servers?

Three options in increasing precision: (1) **short TTL** — accept bounded staleness, simplest and usually enough; (2) **publish/subscribe invalidation** — broadcast a message on write and have every server drop the key; (3) **client-side caching protocols** — the cache server tracks which keys each client holds and pushes an invalidation when they change, which turns coherence from a guess into a protocol.

Q11 How would you size the cache?

Working set (not total dataset) $\times$ entry size $\times$ overhead $\times$ headroom for memory. Then check peak QPS against the per-node throughput ceiling, and check bandwidth — ops/s $\times$ value size. Take the max of the three. Add replicas for HA, not capacity.

Signal: discovering that throughput or bandwidth binds before memory does. See the worked example in §6.2.

Q12 How do you cache a user's news feed / timeline?

Fan-out on write for normal users: when someone posts, push the post ID onto each follower's cached timeline list, capped at ~800 entries. Reads become one range read. But for a celebrity with 200 M followers that write is impossibly expensive — so **hybrid**: fan out for ordinary users, and for celebrities fan-out on read, merging their recent posts into the timeline at request time. This is essentially what Twitter/X does, and it's the canonical answer.

Q13 When would you NOT use a cache?

Low reuse (each key read roughly once — near-zero hit ratio); correctness-critical reads (balances, inventory at checkout, auth revocation); write-heavy/read-light workloads where invalidations exceed hits; and when the backend is already fast — a 2 ms indexed lookup behind a 1 ms cache buys 1 ms and a whole class of consistency bugs. Fix indexing or denormalisation first.

Q14 How do you measure whether your cache is working?

Hit ratio *per namespace* (not just globally — a healthy aggregate hides one broken key space); eviction counters to detect undersizing; p99 latency to detect blocking; DB QPS to confirm the load reduction is real; and staleness incidents from user reports. Track hit ratio alongside eviction rate — a high hit ratio with heavy eviction means you are thrashing and one traffic shift away from a cliff.

Q15 How do you invalidate a browser or CDN cache?

You cannot invalidate a browser cache — only expire it. The answer is **content hashing**: name the asset `/app.a3f19b.js` and cache it for a year, while the HTML that references it is never cached. Change the content → change the URL. A CDN *can* be purged: by URL, or better by **surrogate key** so one product update purges every page that embedded it.

Two header facts: `no-cache` means “store it but revalidate every time” (the one that means don't store is `no-store`), and `stale-while-revalidate` serves the stale copy instantly while refreshing in the background, which removes the latency cliff at expiry.

7.4 Red flags interviewers listen for

Saying this	Says this about you
"Add a cache" before establishing read:write ratio or skew	Pattern-matching, not designing
"Cache everything"	Doesn't understand memory cost or invalidation cost
Updating the cache with the new value on write	Hasn't thought about concurrent writers
No TTL because "we invalidate explicitly"	Hasn't operated a system where an invalidation was missed
Calling the cache an optimisation after sizing the DB for a 99%% hit ratio	Hasn't lived through a cold-cache outage
"Add more shards" as the answer to a hot key	Doesn't understand that one key maps to one shard
Never mentioning what happens when the cache is down	Treats the cache as an optimisation when it has become a dependency
Constant TTLs everywhere	Hasn't seen a synchronised-expiry avalanche
Scanning the whole keyspace to invalidate a prefix	Hasn't run a cache under production load
Treating a cache lock as a correctness guarantee	Missing the efficiency-vs-correctness distinction

7.5 Real systems worth name-dropping

Facebook — Scaling Memcache

The classic paper. Source of **leases** (stampede + stale-set fix in one), the pool separation for different churn rates, and regional invalidation via the MySQL commit log. Cite it for stampedes or invalidation at scale.

Twitter/X — timelines

Hybrid fan-out: precompute timelines on write for ordinary users, merge celebrity posts on read. The canonical answer to the celebrity problem in a feed design.

Netflix — EVCache

A replicated, multi-region cache tier in front of the datastore, sized so a whole zone can be lost. Cite it for "the cache is a dependency" and for warm failover.

MySQL query cache removal

Removed in 8.0 because invalidating on any table write made it a global contention point. The cautionary tale: a cache with a bad invalidation model is worse than no cache.

One-page cheat sheet

The night-before page. If you remember only this, you can still hold a good conversation.

The formulas

$$T_{\text{eff}} = T_c + (1 - H) \times T_d \quad \cdot \quad \text{DB load} = (1 - H) \times \text{QPS}$$

90%→99% hit ratio: latency improves 4×, database load drops 10×. Latency wants 90%; protecting a fragile backend wants 99%+.

The decisions

Question	Default answer
Which pattern?	Cache-aside for reads; DB-write-then-DELETE for writes
Update or delete on write?	Delete. Idempotent, race-tolerant
Which order?	Database first, cache second
TTL?	Always set one, always jitter it, derive it from the business
Eviction policy?	Approximated LRU · LFU when viral spikes pollute the hot set
Which layer?	As close to the user as the staleness budget allows
L1 + L2?	Yes for hot keys: a few-seconds in-process cache in front of the shared tier
Lock for correctness?	Not a cache lock — a DB constraint or a consensus store (etcd/ZooKeeper)

The failure modes

Name	One-line mechanism	Primary fix
Stampede	One hot key expires; N concurrent rebuilds	Probabilistic early expiry / lock / logical expiry
Penetration	Key exists nowhere; every request hits the DB	Cache nulls (short TTL) + Bloom filter
Avalanche	Many keys vanish at once	TTL jitter + HA + circuit breaker
Hot key / celebrity	One key → one shard → one node saturates	L1 cache (reads) + key splitting (writes)
Big key	One huge value blocks the server for its full duration	Split it; cursor-based scans; lazy free
Pollution	Batch scan evicts the hot set	LFU / W-TinyLFU; isolate batch traffic
Cold start	Empty cache after restart/deploy	Prewarm; snapshot restore; ramp traffic

Caching in eight facts

A cache is a bet

Correctness and memory traded for latency and load. Name what you gave up.

Hit ratio is non-linear

90%→99% is 10× less database load. The last few points are the expensive ones.

Delete, don't update

Deletion is idempotent; a write-path update races with concurrent writers.

Database first, cache second

The source of truth commits before the copy is discarded.

Every TTL needs jitter

Constant TTLs synchronise expiry and turn one deploy into an avalanche.

One key lives on one shard

Sharding scales the keyspace, never a single hot key. Split or replicate it.

Working set is not the dataset

Size for the 5–10% that is actually re-read, then add headroom, not the catalogue.

It stops being optional

Once the DB is sized for a 99%% hit ratio the cache is a dependency. Design for its absence.

IF YOU SAY NOTHING ELSE, SAY THIS

"A cache trades correctness and memory for latency and load reduction. I'd use cache-aside with jittered TTLs, delete rather than update on writes, protect the hot keys with a short-TTL local cache, guard rebuilds against stampedes, and design for the cache being unavailable — because once the database is provisioned assuming a 99% hit ratio, the cache stopped being an optimisation and became a dependency."

Sources & further reading — Nishtala et al., *Scaling Memcache at Facebook* (NSDI 2013); Vattani et al., *Optimal Probabilistic Cache Stampede Prevention* (VLDB 2015); Einziger et al., *TinyLFU: A Highly Efficient Cache Admission Policy*; Caffeine design notes on W-TinyLFU; Netflix Tech Blog, *EVCache: Caching at Netflix*; AWS, *Database Caching Strategies*; Alex Xu / ByteByteGo, *A Crash Course in Caching*; Hello Interview, *Caching for System Design Interviews*; DesignGurus, *Caching: strategies, failure modes and trade-offs*; System Design School, *Cache Failure Modes*; MDN, *HTTP caching*; Fastly and Cloudflare docs on surrogate keys and tiered caching; Wikipedia, *Cache stampede*.